

AI Biosensing Platform — Deployment Document (Final Version)
From Local Development → Full Cloud Deployment
Technical Documentation
Piezo Pico-to-Femtotechnology Sensors Inc.
Version 1.0 — September 2026

1. Overview

This document explains how the AI Biosensing Platform moves from **local development** to **full production deployment** using:

- **Render** → Backend hosting
- **Vercel** → Frontend hosting
- **Neon** → Cloud PostgreSQL database
- **Firebase** → Authentication + user management
- **HuggingFace** → ML model hosting
- **Local environment (.env.local)** → Development
- **Cloud environment (.env.production)** → Deployment

2. System Architecture (Simple Diagram) Explanation

Frontend (Next.js / Vercel)

This is the part of the system that people see and use — the website or app interface. When a user clicks a button, fills a form, or views a dashboard, they are interacting with the **frontend**. **Vercel** is the company that hosts this website online so anyone can access it. Think of the frontend as the “face” of the system: clean, friendly, and designed for human interaction.

Backend (FastAPI / Render)

The **backend** is the “brain” behind the scenes. It receives requests from the frontend, processes them, and sends back results. For example, when a user submits a consultation request or asks for device analytics, the backend handles the logic. **Render** is the company that hosts this backend online, making sure it runs 24/7. If the frontend is the face, the backend is the thinking engine that makes everything work.

Database (Neon PostgreSQL)

The **database** is where all important information is stored — things like user profiles, consultation records, device data, and analytics results. **Neon** is a cloud service that keeps

this data safe, organized, and accessible. You can think of it as a secure digital filing cabinet that the backend uses to store and retrieve information whenever needed.

Firestore Auth

Firestore Authentication is the system that manages user login and security. It ensures that only authorized people can access certain parts of the platform. When a user signs in, Firestore verifies their identity and gives them a secure token. This token tells the backend who the user is and what they're allowed to do. In simple terms, Firestore is the “security guard” of the platform.

HuggingFace ML Models

HuggingFace is a service that hosts advanced machine-learning models. These models help the system analyze sensor data, detect patterns, and identify changes or “drift” in device behavior. Instead of running heavy AI computations on your own servers, HuggingFace provides a reliable place to run them. Think of it as an external “AI expert” your backend consults whenever it needs to perform complex analysis. The system architecture (Simple Diagram) is depicted in **Fig.1**.

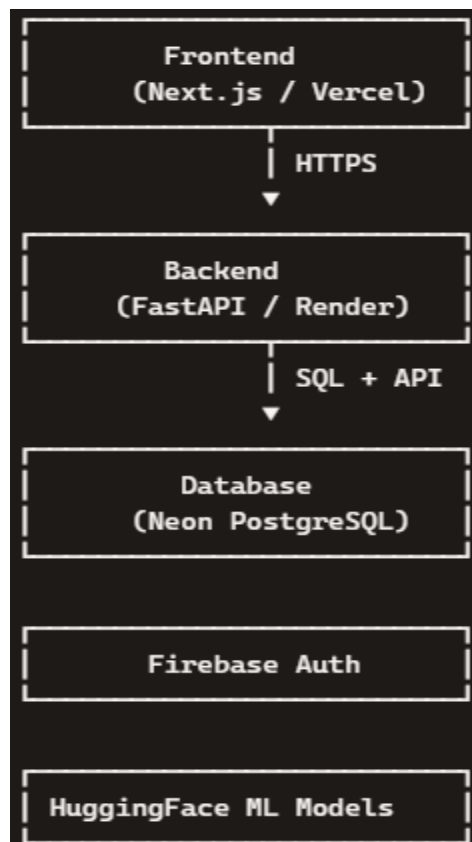


Fig. 1. System Architecture (Simple Diagram)

3. Local Development Setup

3.1 Local Backend

We ran the backend locally using: `uvicorn app.main:app --reload`

Local environment variables were stored in: `.env.local`

Example:

`DATABASE_URL=postgresql://localhost:5432/biosensing`

`HF_API_KEY=xxxx`

`FIREBASE_API_KEY=xxxx`

3.2 Local Frontend

We connected the local frontend to the local backend:

`NEXT_PUBLIC_API_URL=http://localhost:3000`

Frontend ran with: `npm run dev`

This allowed us to simulate:

- consulting workflows
- device analytics
- ML drift
- authentication
- streaming

Everything worked locally before deployment.

4. Cloud Deployment

4.1 Backend Deployment (Render)

Steps

1. Push backend to GitHub
2. Create a Render Web Service
3. Set start command: `uvicorn app.main:app --host 0.0.0.0 --port 10000`
4. Add environment variables in Render:

DATABASE_URL=postgresql+psycopg://<neon-url>

HF_API_KEY=xxxx

FIREBASE_API_KEY=xxxx

RATE_LIMIT_WINDOW=60

RATE_LIMIT_MAX=20

5. Render connects directly to **Neon PostgreSQL**.

Neon Database

Neon provides:

- cloud PostgreSQL
- automatic scaling
- connection pooling

We used the Neon connection string in both:

- local backend
- Render backend

4.2 Frontend Deployment (Vercel)

Steps

1. Push frontend to GitHub
2. Create Vercel project
3. Add environment variables:

NEXT_PUBLIC_API_URL=https://your-render-backend.onrender.com

NEXT_PUBLIC_FIREBASE_API_KEY=xxxx

NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=xxxx

4. Deploy

Frontend → Backend

Vercel communicates with Render through HTTPS.

4.3 Firebase Authentication

Firebase handles:

- user login
- token generation
- secure access to backend endpoints

We used Firebase tokens in:

- consulting endpoints
- device analytics endpoints
- ML drift endpoints

Environment variables were added to both:

- .env.local
- .env.production

4.4 HuggingFace ML Models

HuggingFace hosts:

- ML drift model
- ML training models
- device analytics models

We used:

HF_API_KEY

HF_MODEL_URL

in both local and Render environments.

5. Environment Variables (Complete List)

Backend (.env for Render)

DATABASE_URL=postgresql+psycopg://<neon-url>

HF_API_KEY=xxxx

HF_MODEL_URL=xxxx

FIREBASE_API_KEY=xxxx

FIREBASE_PROJECT_ID=xxxx

RATE_LIMIT_WINDOW=60

RATE_LIMIT_MAX=20

Frontend (.env for Vercel)

NEXT_PUBLIC_API_URL=https://your-backend.onrender.com

NEXT_PUBLIC_FIREBASE_API_KEY=xxxx

NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=xxxx

NEXT_PUBLIC_FIREBASE_PROJECT_ID=xxxx

Local (.env.local)

DATABASE_URL=postgresql://localhost:5432/biosensing

HF_API_KEY=xxxx

NEXT_PUBLIC_API_URL=http://localhost:8000

6. Cleaning the Backend Before Deployment

We performed:

✓ Rate limiting

To prevent overload from ML drift and streaming.

✓ Usage logging

To track endpoint usage.

✓ Token enforcement

To secure consulting and device endpoints.

✓ Removal of local-only code

No more localhost URLs in production.

✓ Removal of unused routers

Only consulting + device analytics remain active.

✓ Removal of local database connections

Everything now uses Neon.

7. Cleaning the Frontend Before Deployment

✓ Removed localhost URLs

✓ Added production API URL

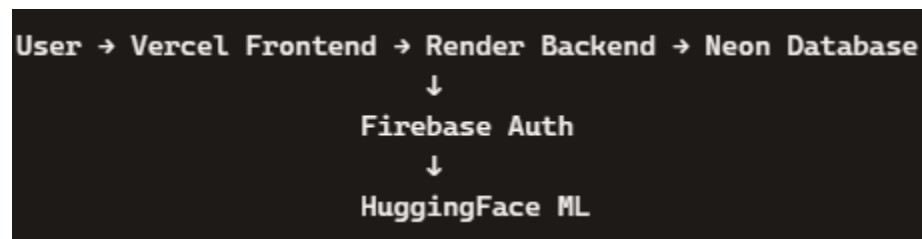
✓ Added Firebase production keys

✓ Removed debug logs

✓ Removed local-only components

✓ Ensured HTTPS-only communication

8. Final Production Flow



Everything is now cloud-based. No local dependencies. No localhost URLs. No local database. No local ML models.

9. Optional: Docker Deployment (Future)

Future containerization:

Backend Dockerfile

FROM python:3.10

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Frontend Dockerfile

```
FROM node:18
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN npm install
```

```
RUN npm run build
```

```
CMD ["npm", "start"]
```

The steps above will allow deployment to:

- AWS
- Azure
- Google Cloud
- DigitalOcean

10. Conclusion

The Company Completed Full Stuck API Architecture:

- a fully documented deployment pipeline
- a clean backend
- a clean frontend
- secure authentication
- cloud database
- cloud ML models
- rate limiting
- production-ready architecture

© **Piezo-Pico to Femtotechnology Sensors Inc.**, All Rights Reserved.

This publication, including all scientific content/documents, biosensing physics, virus classifications, device tables, analytical models, algorithms, and proprietary datasets, is the exclusive intellectual property of **Piezo-Pico to Femtotechnology Sensors Inc.** All materials presented here originate from the company's patented research and internally developed scientific work. No portion of this document may be copied, reproduced, or distributed without prior written authorization from the copyright holder.

All biological information, virus-related data, and analytical structures contained in this work are derived entirely from the company's own home-made datasets and patented methodologies. **Generative AI tools were used solely to assist with collecting publicly available virus mass values and producing complex scientific diagrams**, including virus-mass illustrations, antigen–antibody relationship drawings, and other advanced visual models. AI did not generate or contribute to any proprietary biological data table based on our patented technology, device physics, or algorithmic logic. Its role was limited strictly to visualization and numerical mass extraction from openly accessible scientific sources.

All diagrams, visual models, and scientific illustrations created with AI support remain fully owned by **Piezo-Pico to Femtotechnology Sensors Inc.** as part of its original intellectual property. Any external trademarks, service marks, or product names referenced in this document belong to their respective owners and are used only for descriptive purposes.

Unauthorized use, duplication, or adaptation of any content in these publication/documentations is strictly prohibited and may result in legal action.